MONASH University
Engineering

# ECE2071 Project S1 2026 - Week 4 Activity (Malaysia Campus)

# Task 1. Control of Microcontroller GPIOs using Polling with Delays and Timer-based Interrupts

### Synopsis

In this task, we will continuously toggle the state of a general purpose input/output (GPIO) pin configured as output on the microcontroller using two different methods: polling with delays and timer-based interrupts.

### Required Materials

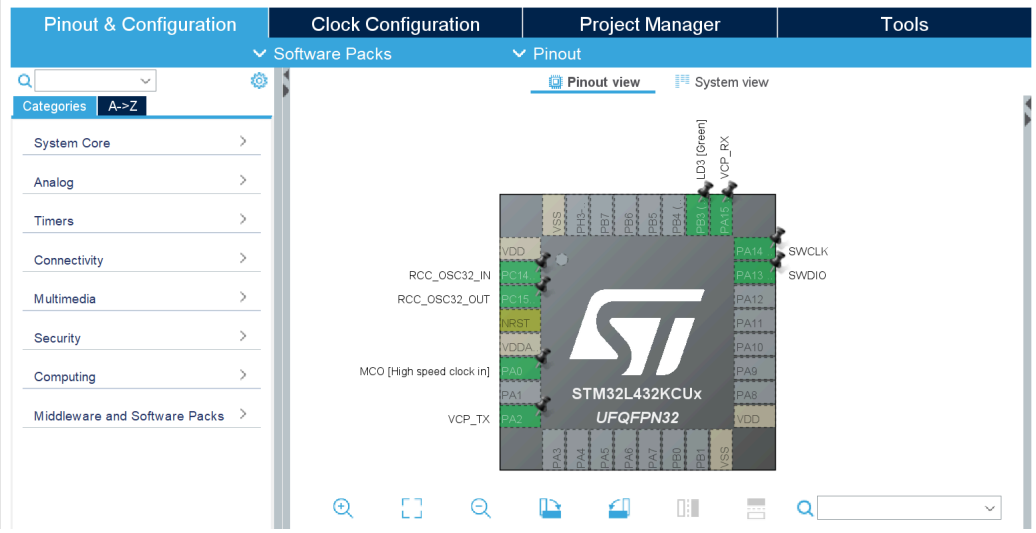
- STM32 Nucleo-L432KC board
- USB Type A-to-Micro B cable
- Personal Computer installed with STM32 CubIDE software
- Resistors
- LEDs

## Task 1(a): Control of Microcontroller GPIOs using Polling with Delays

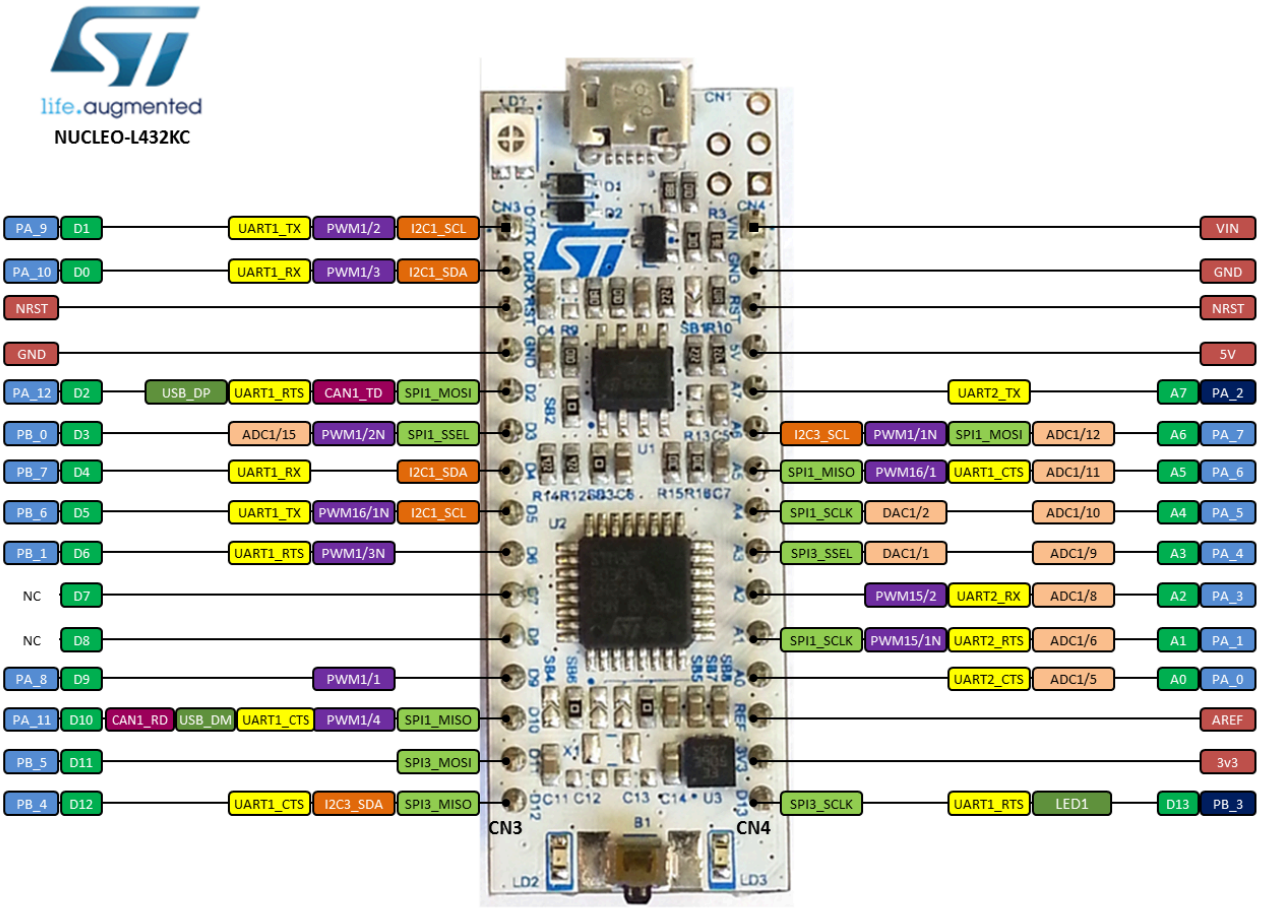
**Step 1:** Connect the STM32 Nucleo-L432KC board to your computer using the USB Type A-to-Micro B cable provided in the kit. The microcontroller should now be powered up (with the red LED LD2 lighted up) and communicating with the computer via ST-LINK (indicated by LD1 in Red). The default program that is downloaded onto the board causes the green user LED (LD3) to continuously blink at a fixed frequency.

**Step 2:** Follow the instructions stated in the Preliminary work for Week 4 Lab document to create your first STM32 project named Project_Week_4.

**Step 3:** Under the Project Explorer tab on the left side of the Cube IDE, click on the file *Project_Week_4.ioc*, which should provide you the Pinout view of the STM32L432KCUx microcontroller microcontroller. Each general purpose input/output pin on the microcontroller has a name that starts with either <PA>, <PB> or <PC> followed by a unique pin number. For instance, Pin PB3 on the microcontroller is internally connected to the green LED LD3 on the board, which is also one of the GPIOs that will be controlled in this task.




Note that the pin names on the STM32L432KCUx microcontroller can be matched to the pin names on the STM32 Nucleo-L432KC board by referring to the following image. For instance, pin PB3 on the microcontroller is connected to Pin D13 on the board.




https://os.mbed.com/platforms/ST-Nucleo-L432KC/

**Step 4:** Click on the file main.c under *Project_Week_4/Core/Src* folder. This will open up a C file where you could write and build your own code, which can then be downloaded onto the microcontroller.

**Step 5:** Initially, we will just try to light up the green user LED LD3 continuously by utilizing the following API:

***void HAL_GPIO_WritePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState)***

which takes in three input arguments:

1. **GPIOx:** where x can be A, B or C depending on whether the name of the pin to be controlled has a prefix of PA, PB or PC, respectively.
2. **GPIO_Pin:** specifies the port bit to be written. This parameter can be any combination of GPIO_PIN_x where x can be (0..15).
3. **PinState:** specifies the value to be written to the selected bit. This parameter can be one of the GPIO_PinState enum values:
    - GPIO_PIN_RESET: to clear the port pin
    - GPIO_PIN_SET: to set the port pin

Since we are trying to light up the on-board green user LED LD3 by setting the pin PB3 on the microcontroller, we will call the *HAL_GPIO_WritePin()* API as follows:

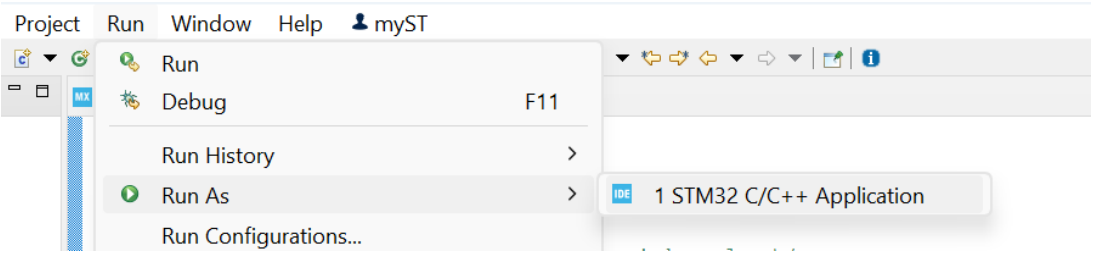
*HAL_GPIO_WritePin(GPIO**B**, GPIO_PIN_**3**, GPIO_PIN_SET)*

Add this line of code in between the `/* USER CODE BEGIN WHILE */` and `/* USER CODE END WHILE */` block comments as shown in the image below.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET);
  /* USER CODE END WHILE */
```

**Any code added outside these user code blocks may be deleted when the program is compiled or if the pin settings are changed. Hence, ensure that user codes are always added within the respective BEGIN and END block comments available at different sections of the program.**

**Step 6:** In order to download the program onto the board, select Run → Run As STM32 C/C++ Application from the toolbar of the STM32Cube IDE as shown in the image below.




**Step 7:** Once the program is downloaded onto the board, the on-board green user LED LD3 should now be continuously lighted up.

**Step 8:** Now, we will modify the program to blink the on-board green user LED LD3 continuously at a frequency of 1 Hz through the use of Polling with delays. In the main.c file, add the following lines of code in between the `/* USER CODE BEGIN WHILE */` and `/* USER CODE END WHILE */` block comments as shown in the image below.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET);
  HAL_Delay(500);
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_RESET);
  HAL_Delay(500);
  /* USER CODE END WHILE */
```

The ***void HAL_Delay (uint32_t Delay)*** API called in the code snippet above allows adding a **time delay equal to the input argument in milliseconds**. Further, adding these lines of code inside the *while (1) loop* causes the code to run sequentially in a repeated manner as long as the board is powered, while reading or writing any GPIO pins, as necessary. This is how GPIO pins of a microcontroller can be controlled through polling with delays.

Now, reduce the lines of code utilized for blinking the on-board green user LED LD3 in the *main.c* file by utilizing the following API instead of the *HAL_GPIO_WritePin()*:

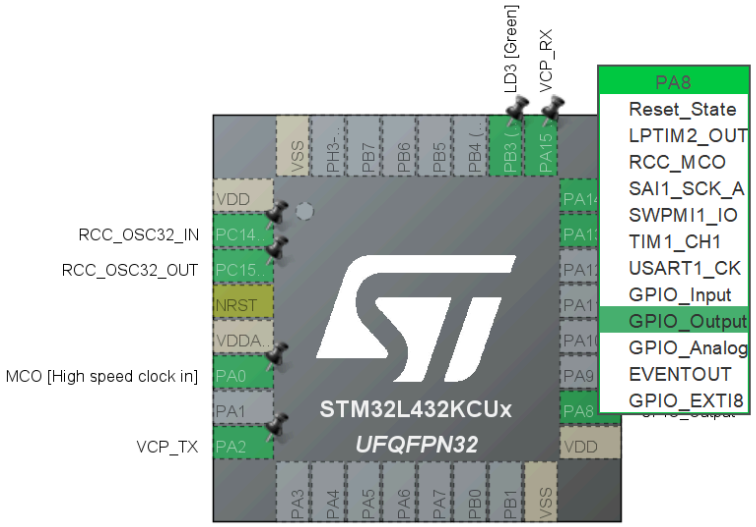
***void HAL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)***

which takes in two input arguments:

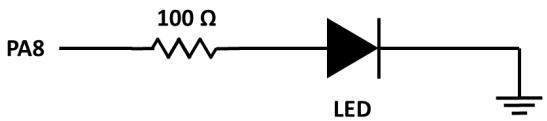
- **GPIOx:** where x can be A, B or C depending on whether the name of the pin to be controlled has a prefix of PA, PB or PC, respectively.
- **GPIO_Pin:** specifies the port bit to be written. This parameter can be any combination of GPIO_PIN_x where x can be (0..15).

Download the program onto the board and verify its functionality.

**Step 9:** While it is useful to utilize the on-board indicator to show program state, it is often more useful to cause changes to external components. To do this you will need to set up a pin to be connected to an external output. Under the Project Explorer tab on the left side of the Cube IDE, click on the file *Project_Week_4.ioc* and then set Pin PA8 as GPIO output as shown in the image below. The pin should turn green on the GUI once this has been performed. Save the *Project_Week_4.ioc* file.




Connect an LED to PA8 on the microcontroller (connected to Pin D9 on the board) as shown in the image below, and then modify the existing code in the *main.c* file to continuously blink both the on-board green user LED LD3 and the LED connected to Pin PA8 at a frequency of 1 Hz. Then, download the program onto the board to verify its functionality.



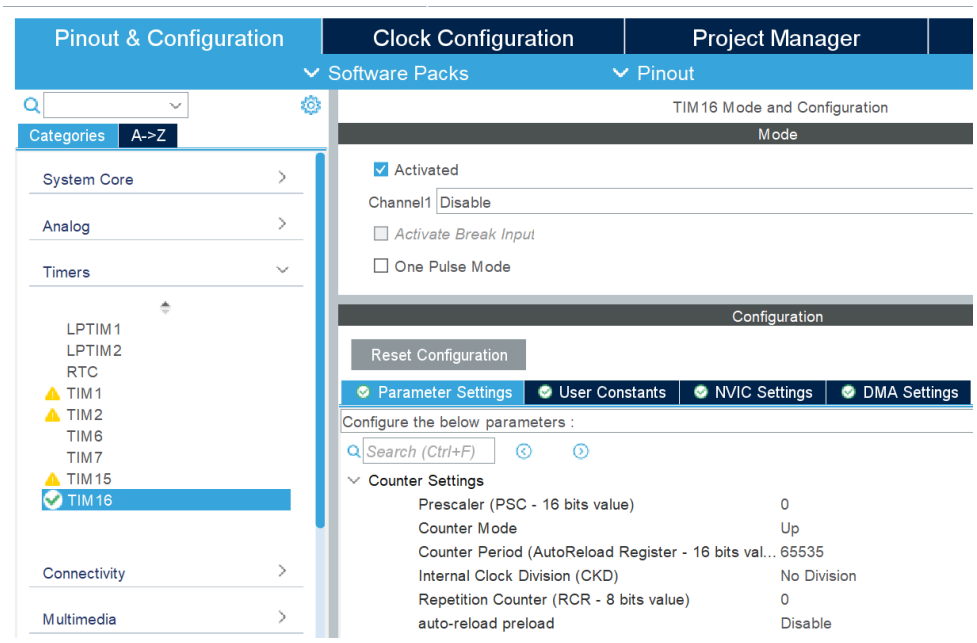
## Task 1(b): Control of Microcontroller GPIOs using Timer-based Interrupts

The blinking sequence implemented in the previous part is functional but limited in its resolution. This is because a polling loop can only be as accurate as the minimum time taken to complete it. Moreover, while the microcontroller is executing the call to *HAL_Delay()* in the previous code, it does not respond to a change in state for any of its input pins, which makes a system controlled by the microcontroller very non-responsive in most situations. This is exactly where interrupts could be deployed instead of polling with delays on a microcontroller to perform periodic control of GPIOs at a much higher resolution and at the same time, improve the responsiveness of the microcontroller to changes in the state of its input pins.

Interrupts are signals on a microcontroller that require immediate action and can be either triggered internally by an in-built peripheral (such as timers, ADCs) or triggered externally by a change in the state of its input pins (such as a proximity sensor detecting an obstacle). When an interrupt is triggered on a microcontroller, normal code execution is temporarily halted and code execution immediately shifts to call a function, termed as interrupt service routine (ISR), written to service the particular interrupt. Once the particular ISR returns, normal code execution resumes at the next instruction from where it had previously stopped before the interrupt signal had occurred.

On the STM32 Nucleo-L432KC board, timers can be used to trigger a variety of interrupts, each supported by a particular callback function. In this part of the task, we will utilize the interrupt signal that is triggered by the microcontroller at the end of each period of a 500 ms timer in order to continuously blink the two LEDs in a similar manner to Task 1 (a). However, with this new approach, regardless of any code that is written inside the *while(1)* loop, and provided another higher priority interrupt is not being handled, we can ensure that both the LEDs will definitely blink continuously at the rate of 1 Hz.

**Step 10:** Under the Project Explorer tab on the left side of the Cube IDE, click on the file *Project_Week_4.ioc*. Click on the "Timers" option under the "Pinout & Configuration" tab to expand it and then click on Timer "TIM16". In order to activate this timer, check the "Activated" option under "TIM16 Mode and Configuration" tab.

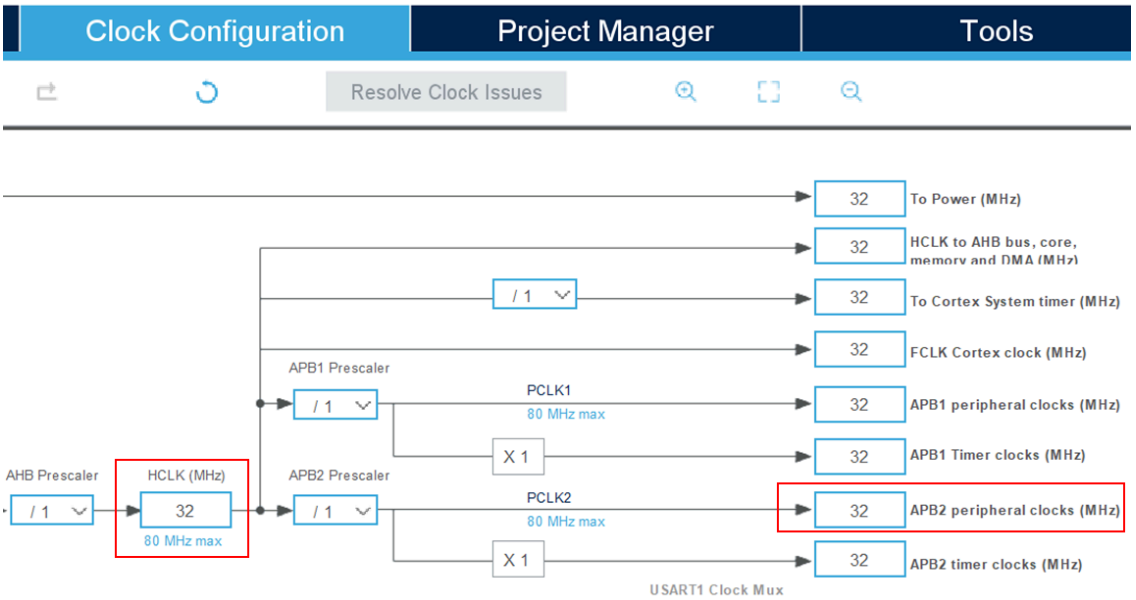


**Step 11:** On the STM32 microcontroller, the in-built hardware timers are actually implemented as counters that increment counts (assuming up-counter) from 0 to a preset value at a preset clock frequency. The preset clock frequency and time period of Timer TIM16 can be calculated using the following expression:

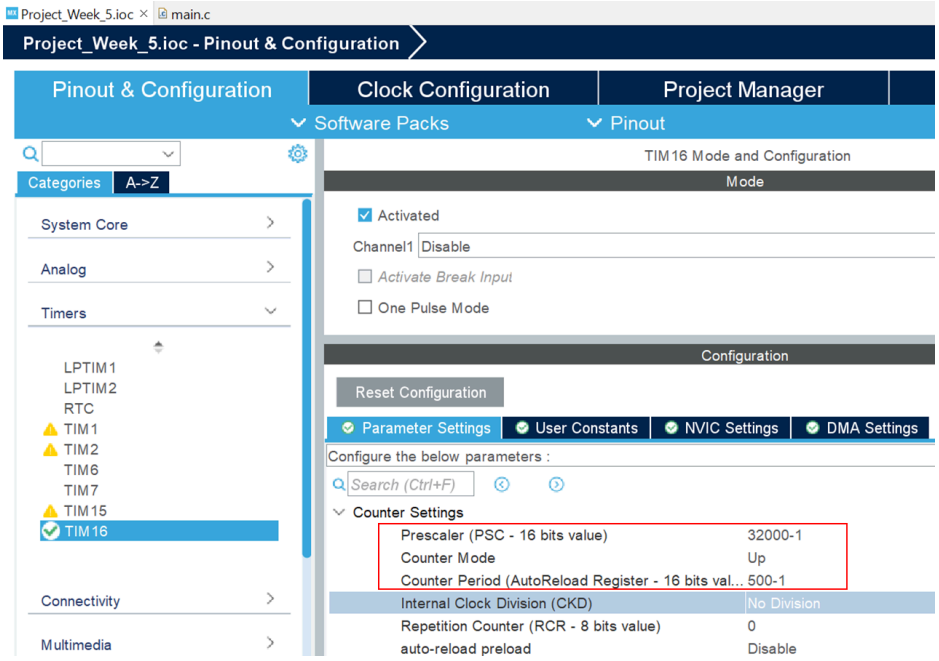
$$Preset\ Clock\ Frequency\ (TIM16) = \frac{Frequency\ of\ internal\ clock\ source\ for\ TIM16\ (APB2)}{Prescaler\ of\ TIM16 + 1}$$

$$Time\ Period\ (TIM16) = \frac{(Counter\ Period + 1)}{Preset\ Clock\ Frequency\ (TIM16)}$$

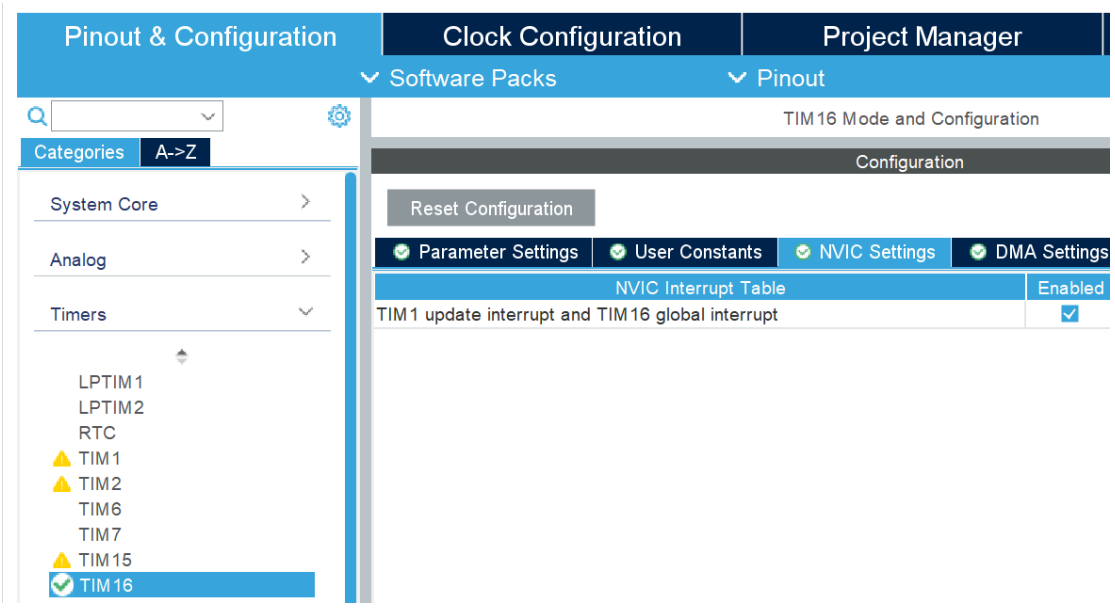
Let's set the period of TIM16 timer to 500 ms in order to blink both the LEDs in Task 1(a) at a frequency of 1 Hz. The default **frequency of the internal clock source for TIM16 (APB2)** can be viewed by clicking on the "Clock Configuration" tab under *Project_Week_4.ioc*. Based on the following image, it could be noted that the default frequency for the APB2 peripheral clock is set as 32 MHz. If necessary, this default setting could be changed to any frequency ≤ 80 MHz by typing the desired frequency (in MHz) in the HCLK field under the same tab.




Go back to the "Pinout & Configuration" tab in the *Project_Week_4.ioc* file and then click on the "TIM16" option again. **Under the parameter settings for Timer TIM16, set the Prescaler to 32000 - 1 and the counter period to 500-1 as shown in the next page.** This should set the timer period of Timer TIM16 to 500 ms. Then, save the *Project_Week_4.ioc* file.




**Step 12:** In order to allow Timer TIM16 to trigger interrupts, under the NVIC (Nested Vectored Interrupt Controller) Settings of TIM16, tick the checkbox "Enabled" for "TIM1 update interrupt and TIM16 global interrupt" as shown in the figure below.




**Step 13:** In the main.c file, add the following line code in between the `/* USER CODE BEGIN 2 */` and `/* USER CODE END 2 */` block comments to start the Timer TIM16 in interrupt mode.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim16);
/* USER CODE END 2 */
```

**Step 14:** In the main.c file, comment out all the code you have previously written for Task 1(a) in between the `/* USER CODE BEGIN WHILE */` and `/* USER CODE END WHILE */` block comments. Thus, the

*while(1)* loop will now not contain any code and by right, if no interrupts are being serviced, the code is stuck in this infinite loop. We will now blink the LEDs continuously by utilizing the following callback function that handles the interrupt triggered by timer TIM16 at the end of each period.

***void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef * htim)***

which takes in one input argument:

- **htim:** TIM handle (for example, &htm16 for timer TIM16)

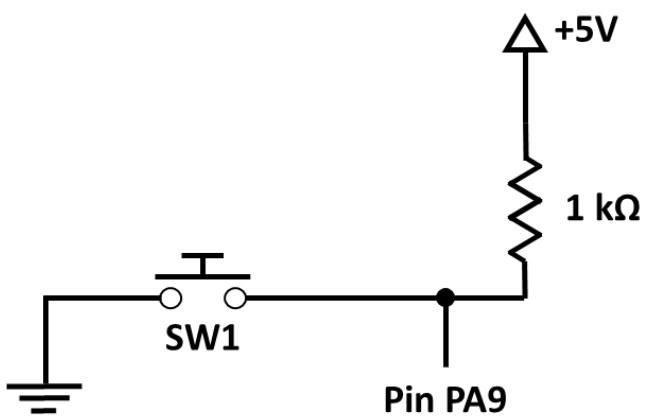
In the *main.c* file, add the following code in between the `/* USER CODE BEGIN 4 */` and `/* USER CODE END 4 */` block comments.

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef * htim)
{
    if (htim == &htim16)
    {
        //insert code to toggle the state of the LEDs here
    }
}
/* USER CODE END 4 */
```

The *if-statement* in the callback function above verifies that the interrupt was triggered by the Timer TIM16 before the state of the LEDs are toggled inside the function.

Insert the code required to toggle the state of the LEDs once within the block of the *if-statement* in the code above. Then, download the program onto the board to verify its functionality.

Connect a pushbutton (provided in the Eleclab kit) to Pin PA9 of the microcontroller as shown in the figure below. Based on the circuit connected, Pin PA9 will be at 5V when the button is not pressed and pulled to ground when the button is pressed.




Modify the project such that it meets the following requirements:

1. If the pushbutton connected to Pin PA9 is not pressed, only the green user LED LD3 should now continuously blink at a frequency of 1 Hz while the LED connected to Pin PA8 should remain OFF.
2. If the pushbutton connected to Pin PA9 is pressed, the green user LED LD3 should immediately turn OFF and the LED connected to Pin PA8 should continuously blink at a frequency of 2 Hz. In order to accomplish the faster blinking rate, you should use a second in-built hardware timer to generate interrupts at the desired new frequency.
3. You are only allowed to write codes inside interrupt callback functions for the two timers utilized for this particular task and thus, the *while(1)* loop inside the main.c file should remain empty.

In order to read the state of a GPIO pin, you could use the following API:
***GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)***
which takes two input arguments:

- **GPIOx:** where x can be A, B or C depending on whether the name of the pin to be controlled has a prefix of PA, PB or PC, respectively.
- **GPIO_Pin:** specifies the port bit to be read. This parameter can be any combination of GPIO_PIN_x where x can be (0..15).

and returns the state of the pin:

- **PinState:** This parameter can be one of the GPIO_PinState enum values:
  - GPIO_PIN_RESET: Logic 0
  - GPIO_PIN_SET: Logic 1

## Understanding the project specifications

The rest of this session will be used to get a better understanding of the prototype to be built in the project, which is due for demonstration during your allocated Week 10 lab session. Refer to the specifications document for this project and have a discussion among your team members to develop a project plan.